Implementing a Feed Forward Neural Network

Image Classification on MNIST and CIFAR-10 using Feed-Forward Neural Networks and Convolutional Neural Networks

Yağmur Geçim
Berkay Bilici

SUPERVISOR

Rebekka Olsson Omslandseter

University of Agder, 2026
Faculty of Engineering and Science
Department of Engineering and Sciences

Master

Abstract

This study focuses on the problem of image classification using two datasets, MNIST and CIFAR-10. The main objective is to evaluate the performance of different models, including a feed-forward neural network (implemented as a multi-layer perceptron), a convolutional neural network, and a logistic regression baseline. The results show that the convolutional neural network achieves the best performance on both datasets, followed by the multi-layer perceptron, while logistic regression performs the worst. This outcome highlights the importance of model architecture in handling complex image data. In particular, the experiments demonstrate that CIFAR-10 is significantly more challenging than MNIST, mainly because of its higher dimensionality and more complex visual structure.

Chapter 1

Introduction

Feedforward neural networks can approximate functions that map input data to output labels, which forms the foundation of image classification tasks. However, for more complex inputs such as images, convolutional neural networks are generally more effective due to their ability to capture spatial relationships [1, pp. 163–165]. In this project, the MNIST and CIFAR-10 datasets are used to compare model performance on relatively simple (MNIST) and more complex (CIFAR-10) image classification tasks. Three models are implemented: logistic regression as a baseline, a multi-layer perceptron (MLP) representing a feedforward neural network, and a convolutional neural network (CNN).
The MNIST dataset is a widely used benchmark dataset in machine learning, consisting of grayscale images of handwritten digits from 0 to 9. Each image is represented as a 28×28 pixel grid and is typically used for classification tasks. Due to its relatively simple structure and low variability, MNIST is commonly used to evaluate basic machine learning models and serves as a starting point for understanding image classification problems [3]. The CIFAR-10 dataset is a more complex image classification dataset consisting of small 32×32 color images belonging to 10 different classes, such as animals and vehicles. Compared to MNIST, CIFAR-10 includes more variability in color, background, and object representation, making it a more challenging dataset [3].

1.1 Experimental Setup

After completing the code in Google Colab (Python), first the code is run in M1 chip Macbook and it took 26 minutes since it was using CPU to run. Approximately 7 minutes for MNIST dataset and 19 minutes for CIFAR dataset. It was known that CIFAR is a more complex dataset and it might take more time to run, but the total of 26 minutes was shocking. After searching and discussing it with AI tools, we changed the code to use GPU instead of CPU if available, it took 8 minutes in total. So basically, the model is trained on GPU when available to improve computational efficiency and reduce training time.

```
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if ...
   torch.backends.mps.is_available() else "cpu")
print("used device:", device)
```

1.2 Training Setup

The model is trained for 10 epochs, where each epoch represents one complete pass over the training dataset. This value is chosen as a balance between sufficient learning and avoiding overfitting. If the number of epochs is too low (e.g., 1 epoch), the model does not have enough time to learn meaningful patterns, leading to underfitting and low accuracy. On the other hand, if the number of epochs is too high (e.g., 100 epochs), the model may start to memorize the training data, resulting in overfitting and reduced generalization performance on unseen data [1].

The learning rate is set to 0.001, which is a commonly used value for the Adam optimizer as it provides stable and efficient convergence [5]. A higher learning rate may cause the model to overshoot optimal values and fail to converge, while a very small learning rate can make training extremely slow and inefficient.
A batch size of 64 is used to balance computational efficiency and learning stability. Smaller batch sizes introduce more noise in gradient updates, which can improve generalization but slow down training. In contrast, larger batch sizes provide faster computation but may lead to poorer generalization performance [1].

1.3 Train / Validation / Test Split

The dataset is divided into three subsets: training, validation, and test sets. The training set is used to learn the model parameters, while the validation set is used during training to tune hyper-parameters and monitor generalization performance. The test set is reserved for final evaluation and is used only once to assess how well the model performs on unseen data [1].

1.4 Data Preprocessing

In this project, the input data is first normalized by scaling pixel values to the range [0, 1], which improves numerical stability and accelerates the training process [1]. Since feedforward neural networks (MLPs) require vectorized inputs, the image data is flattened into one-dimensional vectors (784 features for MNIST and 3072 for CIFAR-10) before being passed to fully connected layers [1]. In contrast, for Convolutional Neural Networks (CNNs), the data is reshaped into its original format, allowing the model to preserve spatial information and learn meaningful visual patterns [1], [3].
This preprocessing step highlights a key difference between the datasets and models: while MNIST consists of simple grayscale images that can still be handled reasonably well after flattening, CIFAR-10 contains more complex color images with richer spatial structure. Therefore, preserving the image format through reshaping becomes especially important for CNN models when working with CIFAR-10 [3].

Chapter 2

Theory

2.1 Logistic Regression

Logistic Regression is included in this project as a baseline model to provide a reference point for performance comparison. Since it is a simple linear model, it allows us to evaluate how much performance improvement is achieved by more complex models such as MLP and CNN. This comparison helps highlight the importance of non-linear learning and advanced architectures in image classification tasks [1].

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
log_preds = log_model.predict(X_test)

log_acc = accuracy_score(y_test, log_preds)
print("\nLogistic Regression:")
print(classification_report(y_test, log_preds))
```

The Logistic Regression model achieves an overall accuracy of 92 percent on the MNIST dataset, indicating that even a simple linear model can perform reasonably well on this relatively simple dataset. However, performance varies across classes, with higher scores for simpler digits such as “1” and “0”, and lower scores for more complex or visually similar digits such as “5” and “8”. The precision and recall values are generally balanced, suggesting that the model does not exhibit strong bias toward specific classes.

```
Logistic Regression:
              precision    recall  f1-score   support

           0       0.95      0.96      0.96      1029
           1       0.96      0.97      0.97      1144
           2       0.91      0.90      0.90      1027
           3       0.90      0.89      0.90      1059
           4       0.92      0.93      0.92      1018
           5       0.86      0.88      0.87      1022
           6       0.94      0.96      0.95      1032
           7       0.93      0.92      0.93      1105
           8       0.89      0.87      0.88      1051
           9       0.89      0.90      0.90      1013

    accuracy                           0.92     10500
   macro avg       0.92      0.92      0.92     10500
weighted avg       0.92      0.92      0.92     10500
```

The Logistic Regression model performs significantly worse on the CIFAR-10 dataset, achieving only 34 percent accuracy compared to 92 percent on MNIST. This highlights the increased complexity of CIFAR-10, which consists of colored images with more variability in object shapes, backgrounds, and textures. As a linear model, Logistic Regression is unable to capture the complex patterns present in the dataset. The low and inconsistent precision, recall, and F1-scores across all classes further confirm that the model struggles to generalize. These results demonstrate that simple linear models are insufficient for complex image classification tasks such as CIFAR-10.

```
Logistic Regression:
              precision    recall  f1-score   support

           0       0.42      0.41      0.42       309
           1       0.39      0.39      0.39       314
           2       0.26      0.29      0.27       298
           3       0.22      0.25      0.23       301
           4       0.27      0.28      0.27       291
           5       0.31      0.25      0.28       307
           6       0.34      0.33      0.33       297
           7       0.38      0.37      0.38       300
           8       0.42      0.41      0.41       294
           9       0.35      0.38      0.37       289

    accuracy                           0.34      3000
   macro avg       0.34      0.34      0.34      3000
weighted avg       0.34      0.34      0.34      3000
```

2.2 Feed Forward Neural Network (FFNN)

Feedforward neural networks, also known as multi-layer perceptrons (MLPs), are fundamental models in deep learning. In these models, information flows in one direction, from the input x through hidden layers to the output y, without any feedback connections. In this project, the implemented MLP follows this structure by processing input images through multiple fully connected layers to produce classification results [1].

2.2.1 How does it work?

Training a Feedforward Neural Network is an iterative process of optimizing neuron weights to minimize the discrepancy between predicted and actual outputs. The cycle begins with forward propagation, where input data travels through the network layers to generate a prediction. Following this, the loss calculation step uses a function, such as Cross-Entropy Loss for classification, to quantify the model's error. Finally, through backpropagation, this error is sent backward through the architecture to calculate gradients, allowing an optimizer like gradient descent to adjust the weights accordingly. By repeating these steps, the model continuously refines its parameters to improve accuracy and reduce overall loss [6].

2.2.2 Layers

In this project, the implemented feedforward neural network consists of three hidden layers with 256, 128, and 64 neurons, followed by an output layer with 10 neurons for classification. Here is the code sample from MNIST from Google Colab Python the project that shows there is 1 input layer, 3 hidden layer, 1 output layer:

```
nn.Linear(input_size, 256)
nn.Linear(256, 128)
nn.Linear(128, 64)
nn.Linear(64, 10)
```

The architecture is organized into three functional stages: The input layer flattens the multidimensional image data (784 features for MNIST, 3072 for CIFAR) into a vector. The hidden layers act as feature extractors where the actual learning occurs by capturing patterns. There are 3 hidden layers in this project. Finally, in the output layer the model decides which number is it or what is in the picture; since there are 10 numbers (zero to nine) in MNIST dataset there are 10 nodes and similarly since in the CIFAR dataset there are 10 nodes.
What happens if we use zero, one, two, three, or more hidden layers? When there is no hidden layer, the model reduces to logistic regression and can only learn linear relationships. However,

real-world data is usually non-linear, which limits the performance of such models. Based on our experiments, this approach performs relatively better on the MNIST dataset compared to CIFAR-10, but still worse than deeper models. This approach is added to our code project to see how the baseline model works and how precision, recall and f1 score changes between models by using the same dataset. The results of these approaches will be discussed and compared in Baseline Model and Results parts.

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
log_preds = log_model.predict(X_test)

log_acc = accuracy_score(y_test, log_preds)
print("\nLogistic Regression:")
print(classification_report(y_test, log_preds))
```

When one hidden layer is introduced, the model becomes capable of learning non-linear relationships, allowing it to capture basic patterns in the data. Adding a second hidden layer further improves the model's ability to learn more complex patterns and representations.
In this assignment, we used a model with three hidden layers. This choice was based on experimental observations showing that deeper architectures can learn more meaningful features and improve overall performance. In particular, this structure provides better feature extraction and helps control overfitting when combined with techniques such as dropout [1].
In general, increasing the number of hidden layers allows the model to learn more complex representations [1]. However, beyond a certain point, deeper models may suffer from overfitting and become harder to train [1], [2]. Therefore, in this project, a three-hidden-layer architecture was selected as a balance between model complexity and generalization performance, especially for more challenging datasets such as CIFAR-10.

2.2.3 Activation and Dropout

The architecture of feedforward neural networks incorporates hidden layers, thereby requiring the deliberate choice of activation functions for calculating hidden layer outputs [1, p. 171]. While writing MLP class in code, activation function, in other words ReLU() is added to code to introduce non-linearity, allowing the model to learn complex patterns. Without activation, model makes linear transformation, which makes no difference from Logistic Regression that we implemented to show the basic model [1].

```
        self.model = nn.Sequential(
            nn.Linear(input_size, 256),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(128, 64),
            nn.ReLU(),

            nn.Linear(64, 10)
        )
```

Dropout is an effective way to regularize models by creating new input data through the addition of noise [1, p. 241]. As demonstrated in the implementation, Dropout layers are integrated following the ReLU activation functions. This approach is crucial to prevent the model from memorizing the training dataset, a common cause of overfitting. By randomly deactivating a subset of neurons during each training iteration (specifically 30 percent in this project, given the 0.3 dropout rate) the model is forced to learn more robust features. This process effectively creates a different network

architecture at each step, enhancing the model's ability to generalize rather than simply overfitting to the training data [4].
Briefly, in each layer, the model first performs a linear transformation, then applies a ReLU activation to introduce non-linearity, and finally uses dropout to reduce overfitting.

2.2.4 Training setup

The model is trained using the cross-entropy loss function, which is widely used for multi-class classification tasks [1]. The cross-entropy loss function measures the difference between the predicted class probabilities and the true labels, making it suitable for multi-class classification tasks [1]. It penalizes incorrect predictions more strongly, guiding the model to improve its accuracy during training.

```
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
```

The optimization is performed using the Adam optimizer with a learning rate of 0.001, as it provides efficient and adaptive parameter updates [5]. The Adam optimizer was utilized to iteratively update the network's weights by minimizing the loss function as wanted in the assignment document and it was selected for its adaptive learning rate capabilities. Training is conducted using mini-batches of size 64 to improve computational efficiency and stability [1].

```
def create_loaders(X_train, y_train, X_val, y_val, batch_size=64):
    train_ds = TensorDataset(X_train, y_train)
    val_ds = TensorDataset(X_val, y_val)

    return DataLoader(train_ds, batch_size=batch_size, shuffle=True), \
           DataLoader(val_ds, batch_size=batch_size)
```

The model is trained for multiple epochs (epochs = 10), where each epoch represents one full pass over the training data. In addition, a validation set is used during training to monitor the model's generalization performance and detect potential overfitting [1].

2.2.5 Data Handling

The dataset is divided into training, validation, and test sets using a 70-15-15 split. The training set is used to train the model, while the validation set is used during training to monitor performance and tune hyperparameters. The test set is used only once at the end to evaluate the final performance of the model on unseen data [1].

```
 X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3)
    X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, ...
        test_size=0.5)
```

In the feedforward neural network (MLP) implementation, the input data is preprocessed by normalizing pixel values to the range [0,1] to improve training stability [1].

```
mnist = fetch_openml('mnist_784', version=1)
X = mnist.data.astype('float32').values / 255.0
y = mnist.target.astype(int).values
```

The MNIST and CIFAR-10 datasets are provided in a flattened format, where each image is represented as a one-dimensional vector. This representation is required for fully connected neural networks, which operate on vectorized inputs [1]. The dataset is then split into training, validation, and test sets to ensure proper model evaluation and generalization [2]. Finally, the data is converted into tensor format to enable efficient numerical computation and support gradient-based optimization in neural network frameworks [1], [3].

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
```

Finally, data loaders are used to divide the dataset into mini-batches, enabling efficient and stable training [1].

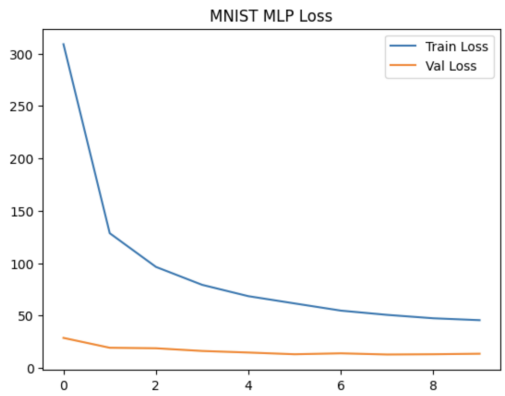
2.2.6 Results of FFNN

For MNIST

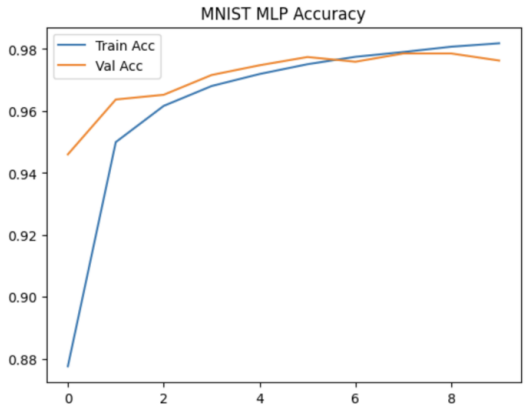
The MLP model on the MNIST dataset demonstrates rapid learning, with training accuracy increasing from 0.87 to above 0.96 within the first three epochs. Both training and validation accuracies continue to improve and stabilize around 0.97–0.98, indicating that the model successfully learns the underlying patterns. The small gap between training and validation accuracy suggests that overfitting is minimal. However, a slight decrease in validation accuracy in the final epochs indicates the beginning of overfitting. Overall, the results show that a moderate number of epochs is sufficient for this relatively simple dataset.

```
Epoch 1: Train Acc=0.8776, Val Acc=0.9460
Epoch 2: Train Acc=0.9500, Val Acc=0.9637
Epoch 3: Train Acc=0.9617, Val Acc=0.9652
Epoch 4: Train Acc=0.9681, Val Acc=0.9716
Epoch 5: Train Acc=0.9719, Val Acc=0.9748
Epoch 6: Train Acc=0.9751, Val Acc=0.9774
Epoch 7: Train Acc=0.9775, Val Acc=0.9759
Epoch 8: Train Acc=0.9790, Val Acc=0.9786
Epoch 9: Train Acc=0.9808, Val Acc=0.9786
Epoch 10: Train Acc=0.9819, Val Acc=0.9763
```

The loss curves show a rapid decrease in both training and validation loss during the first few epochs, indicating efficient learning. After approximately 4–5 epochs, the decrease becomes more gradual, suggesting that the model is approaching convergence. The validation loss remains stable and does not increase, indicating that overfitting is well controlled.



Similarly, the accuracy curves show a sharp increase in both training and validation accuracy, reaching approximately 98 percent. The small gap between training and validation accuracy suggests that the model generalizes well. A slight decrease in validation accuracy in the final epochs indicates the early signs of overfitting, but overall performance remains stable.

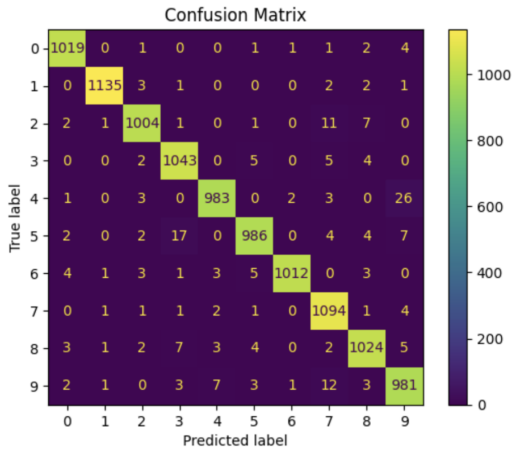


The MLP model achieves a high classification accuracy of 98 percent on the MNIST dataset, significantly outperforming the Logistic Regression baseline. The performance is consistent across all classes, with most F1-scores ranging between 0.97 and 0.99. This indicates that the model successfully captures complex patterns in the data. The precision and recall values are well balanced, suggesting that the model does not suffer from class bias. Slightly lower performance is observed in classes such as “9” and “5”, which may be due to visual similarities with other digits. Overall, the results demonstrate that MLP is highly effective for relatively simple image classification tasks such as MNIST.

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1029
1	1.00	0.99	0.99	1144
2	0.98	0.98	0.98	1027
3	0.97	0.98	0.98	1059
4	0.98	0.97	0.98	1018
5	0.98	0.96	0.97	1022
6	1.00	0.98	0.99	1032
7	0.96	0.99	0.98	1105
8	0.98	0.97	0.97	1051
9	0.95	0.97	0.96	1013
accuracy			0.98	10500
macro avg	0.98	0.98	0.98	10500
weighted avg	0.98	0.98	0.98	10500

Macro F1: 0.9789820899958265
Weighted F1: 0.9791583790151892

The confusion matrix shows that the MLP model correctly classifies the majority of the samples, as indicated by the strong diagonal values. However, some misclassifications are observed between visually similar digits, such as “4” and “9”, “5” and “3”, and “9” and “7”. These errors suggest that while the model performs well overall, it still struggles to distinguish between digits with similar shapes. This limitation is expected, as MLP models do not explicitly capture spatial relationships between pixels.

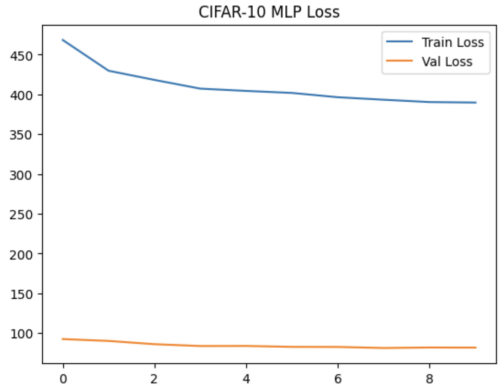


For CIFAR-10

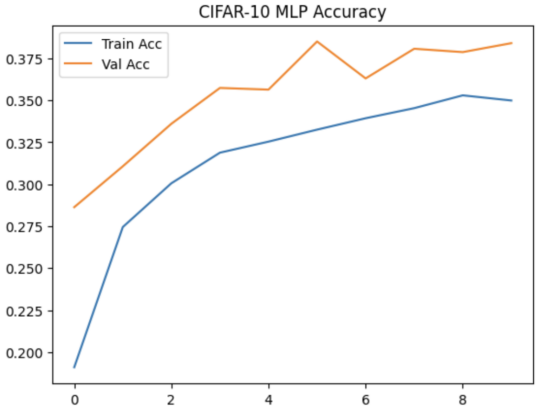
The MLP model shows gradual improvement on the CIFAR-10 dataset, with training accuracy increasing from 19 percent to approximately 35 percent. However, the overall performance remains significantly lower compared to MNIST, indicating that the model struggles to learn complex patterns in CIFAR-10. The validation accuracy is slightly higher than training accuracy, suggesting the presence of underfitting rather than overfitting. Additionally, the accuracy stabilizes after a few epochs, indicating that the model has reached its learning capacity. These results highlight that while MLP can capture some patterns, it is insufficient for complex image data such as CIFAR-10.

```
Epoch 1: Train Acc=0.1910, Val Acc=0.2863
Epoch 2: Train Acc=0.2745, Val Acc=0.3107
Epoch 3: Train Acc=0.3006, Val Acc=0.3360
Epoch 4: Train Acc=0.3188, Val Acc=0.3573
Epoch 5: Train Acc=0.3254, Val Acc=0.3563
Epoch 6: Train Acc=0.3325, Val Acc=0.3850
Epoch 7: Train Acc=0.3393, Val Acc=0.3630
Epoch 8: Train Acc=0.3453, Val Acc=0.3807
Epoch 9: Train Acc=0.3529, Val Acc=0.3787
Epoch 10: Train Acc=0.3499, Val Acc=0.3840
```

The loss curves for the CIFAR-10 dataset show a slow and limited decrease in both training and validation loss, indicating that the model struggles to learn complex patterns. Unlike the MNIST dataset, where loss decreases rapidly, the CIFAR-10 loss remains relatively high throughout training.



Similarly, the accuracy curves demonstrate only a modest improvement, with validation accuracy reaching approximately 38 percent. The validation accuracy being consistently higher than the training accuracy suggests underfitting rather than overfitting. Additionally, the performance stabilizes after a few epochs, indicating that the model has reached its capacity. These results highlight that MLP is insufficient for complex datasets like CIFAR-10.

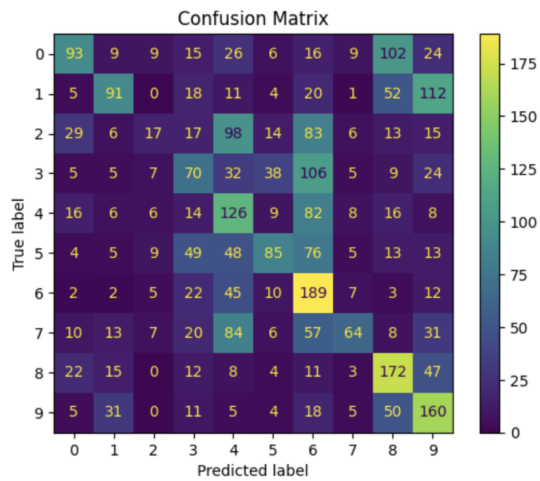


The MLP model achieves only 36 percent accuracy on the CIFAR-10 dataset, indicating poor performance compared to MNIST. The class-wise F1-scores vary significantly, ranging from as low as 0.09 to 0.47, showing that the model struggles to consistently learn meaningful patterns across classes. In particular, some classes such as "2" have extremely low recall, indicating that the model fails to correctly identify many true samples.

	precision	recall	f1-score	support
0	0.49	0.30	0.37	309
1	0.50	0.29	0.37	314
2	0.28	0.06	0.09	298
3	0.28	0.23	0.26	301
4	0.26	0.43	0.33	291
5	0.47	0.28	0.35	307
6	0.29	0.64	0.40	297
7	0.57	0.21	0.31	300
8	0.39	0.59	0.47	294
9	0.36	0.55	0.44	289
accuracy			0.36	3000
macro avg	0.39	0.36	0.34	3000
weighted avg	0.39	0.36	0.34	3000

Macro F1: 0.3373894112264548
Weighted F1: 0.33714171205104354

The confusion matrix for the CIFAR-10 dataset shows that the MLP model struggles significantly with classification. Unlike the MNIST case, where most predictions lie along the diagonal, the CIFAR-10 matrix displays widespread misclassification across multiple classes. Many samples are incorrectly predicted as other categories, indicating that the model fails to distinguish between different object classes.
Notably, strong confusion is observed between several class pairs, such as class "0" being predicted as "8", class "1" as "9", and class "3" as "6". These errors suggest that the model cannot effectively capture meaningful features from the data. This limitation arises because MLP models operate on flattened inputs and do not preserve spatial relationships, which are essential for complex image datasets like CIFAR-10.



2.3 MLP (Multi-Layer Perceptron)

A Multi-Layer Perceptron (MLP) is a type of feedforward neural network that consists of multiple hidden layers. Unlike simple models without hidden layers, MLPs are capable of learning non-linear relationships by applying activation functions between layers. This allows the model to capture more complex patterns in the data and improve classification performance [1].
In this project, an MLP is used because it can model non-linear relationships between input features and output classes, unlike simpler models such as logistic regression. This makes it more suitable for image classification tasks, where the data contains complex patterns [1]. As discussed in Section 2.1, simpler models may achieve acceptable performance on the MNIST dataset due to its relatively low complexity; however, deeper models such as MLP provide better performance on both datasets, especially on the more complex CIFAR-10 dataset. Since an MLP is a multi-layered extension of a feedforward neural network, its main structure and properties have already been explained in Section 2.2.

2.4 Convolutional Neural Networks (CNN)

In feedforward neural networks such as Multi-Layer Perceptrons (MLPs), input images are typically flattened into one-dimensional vectors (e.g., a 28×28 image is converted into a vector of size 784). While this allows the data to be processed by fully connected layers, it removes the spatial relationships between pixels, limiting the model's ability to capture local patterns such as edges and textures [1]. In contrast, Convolutional Neural Networks (CNNs) preserve the two-dimensional structure of images and are specifically designed to learn spatial and hierarchical features.
CNNs operate by applying convolutional layers, where small filters (kernels) slide over the image to extract local features, followed by pooling layers that reduce spatial dimensions while retaining important information [7], [8].
For example, in our implementation, the convolutional layers are defined in lines 46 and 50 using Conv2d, where the value 3 represents the kernel size. In lines 48 and 52, pooling operations are applied using MaxPool2d, which reduces the spatial dimensions of the images. In this block, feature extraction is performed through convolution and pooling operations. The extracted features are later passed to fully connected layers for classification.

```
self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
```

The extracted features are then passed to fully connected layers for classification. This architecture enables CNNs to learn increasingly complex representations, making them more effective for image classification tasks, especially for more complex datasets such as CIFAR-10 [1], [9].
In the forward function, the input is first processed through convolutional layers (self.conv(x)), then flattened into a one-dimensional vector using view(), and finally passed to fully connected layers (self.fc(x)) to produce the final classification output.

```
   def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

Most importantly, CNNs preserve the spatial structure of images by processing them as two-dimensional matrices. In contrast, MLP models require flattened inputs, which removes spatial

information. In this project, this difference is implemented by reshaping the MNIST dataset (line 256) and the CIFAR-10 dataset (line 261) into their original image formats before applying CNN models.

```
    if not is_cifar:
        X_train_cnn = X_train.reshape(-1,1,28,28)
        X_val_cnn = X_val.reshape(-1,1,28,28)
        X_test_cnn = X_test.reshape(-1,1,28,28)
        model = CNN_MNIST()
    else:
        X_train_cnn = X_train.reshape(-1,3,32,32)
        X_val_cnn = X_val.reshape(-1,3,32,32)
        X_test_cnn = X_test.reshape(-1,3,32,32)
```

2.4.1 Why CNN is More Effective for Image Data

Convolutional Neural Networks (CNNs) are more effective than fully connected networks such as MLPs for image data because they preserve spatial structure and utilize local connectivity and weight sharing. In CNNs, filters are applied over small regions of the image, allowing the model to focus on meaningful local patterns such as edges and textures while significantly reducing the number of parameters [1]. This not only improves computational efficiency but also enhances generalization performance. In contrast, MLPs treat each pixel independently after flattening, which results in loss of spatial information and weaker performance on image-based tasks [1], [7].
In our implementation, this is achieved using convolutional layers (nn.Conv2d) and pooling layers (nn.MaxPool2d), which extract spatial features before passing them to fully connected layers.

2.4.2 CNN Performance on CIFAR-10 Dataset

The advantages of CNNs become more evident when working with complex datasets such as CIFAR-10. Unlike MNIST, which contains simple grayscale digit images, CIFAR-10 includes color images with varying backgrounds, object orientations, and textures. These characteristics require models to capture more complex spatial patterns. CNNs are particularly well-suited for this task because they can learn hierarchical feature representations, starting from low-level features such as edges and progressing to higher-level object structures [1], [9].
In our project, the CIFAR-10 dataset is reshaped into a 3D format (reshape(-1, 3, 32, 32)), allowing CNN models to process color channels and spatial relationships effectively, which significantly improves classification performance compared to MLP-based approaches.

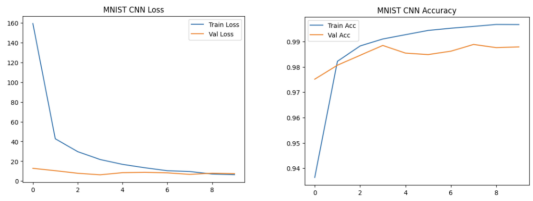
2.4.3 Results

For MNIST

The CNN model achieves near-perfect performance on the MNIST dataset, reaching approximately 99 percent accuracy. The model converges very quickly, achieving high accuracy within the first few epochs. Both training and validation accuracy remain consistently high and close to each other, indicating strong generalization with minimal overfitting. Compared to the MLP model, CNN provides slightly better performance due to its ability to capture spatial features such as edges and local patterns. These results confirm that CNN is highly effective for image classification tasks, even for relatively simple datasets like MNIST.

```
Epoch 1: Train Acc=0.9363, Val Acc=0.9751
Epoch 2: Train Acc=0.9822, Val Acc=0.9807
Epoch 3: Train Acc=0.9883, Val Acc=0.9846
Epoch 4: Train Acc=0.9910, Val Acc=0.9885
Epoch 5: Train Acc=0.9927, Val Acc=0.9854
Epoch 6: Train Acc=0.9944, Val Acc=0.9849
Epoch 7: Train Acc=0.9953, Val Acc=0.9862
Epoch 8: Train Acc=0.9960, Val Acc=0.9889
Epoch 9: Train Acc=0.9968, Val Acc=0.9876
Epoch 10: Train Acc=0.9967, Val Acc=0.9879
```

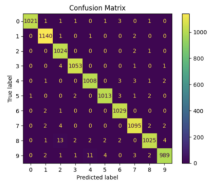
The CNN model demonstrates rapid convergence during the initial epochs, as evidenced by the sharp decrease in training loss and the corresponding increase in validation accuracy. This indicates that the model quickly captures the fundamental patterns in the MNIST dataset. However, after approximately 3–4 epochs, validation performance begins to plateau while training accuracy continues to improve. This suggests that the model starts to overfit slightly, focusing more on training-specific details rather than improving generalization. Despite this, the gap between training and validation accuracy remains small, indicating strong overall generalization performance.



The CNN model achieves near-perfect performance on the MNIST dataset, with an overall accuracy of 99 percent. Both precision and recall values are consistently high across all classes, resulting in F1-scores close to 1. The confusion matrix further confirms this performance, showing that most predictions lie on the diagonal, indicating correct classifications. Misclassifications are minimal and mainly occur between visually similar digits such as 8 and 9. This demonstrates that CNNs are highly effective in capturing spatial patterns in simple image datasets like MNIST.

	precision	recall	f1-score	support
0	1.00	0.99	1.00	1029
1	0.99	1.00	0.99	1144
2	0.98	1.00	0.99	1027
3	0.99	0.99	0.99	1059
4	0.99	0.99	0.99	1018
5	0.99	0.99	0.99	1022
6	0.99	1.00	0.99	1032
7	0.99	0.99	0.99	1105
8	0.99	0.98	0.98	1051
9	0.99	0.98	0.98	1013
accuracy			0.99	10500
macro avg	0.99	0.99	0.99	10500
weighted avg	0.99	0.99	0.99	10500

Macro F1: 0.9901290283979666
Weighted F1: 0.9901841985932917

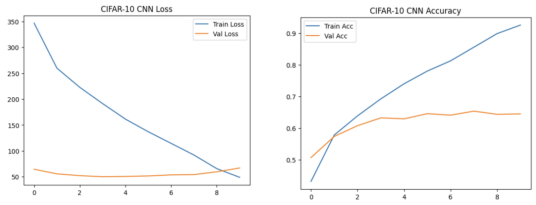


For CIFAR-10

Unlike the MNIST case, the CNN model struggles to generalize on the CIFAR-10 dataset. While training accuracy increases significantly from 43 percent to over 92 percent, validation accuracy plateaus around 65 percent afteer a few epochs. This creates a large gap between training and validation performance, indicating severe overfitting. The model continues to learn training-specific patterns but fails to improve generalization. This behavior highlights the increased complexity of CIFAR-10 compared to MNIST and shows that the current CNN architecture is not sufficiently powerful to capture the full variability of the dataset.

```
Epoch 1: Train Acc=0.4313, Val Acc=0.5067
Epoch 2: Train Acc=0.5782, Val Acc=0.5737
Epoch 3: Train Acc=0.6387, Val Acc=0.6077
Epoch 4: Train Acc=0.6926, Val Acc=0.6323
Epoch 5: Train Acc=0.7404, Val Acc=0.6293
Epoch 6: Train Acc=0.7806, Val Acc=0.6457
Epoch 7: Train Acc=0.8130, Val Acc=0.6410
Epoch 8: Train Acc=0.8561, Val Acc=0.6537
Epoch 9: Train Acc=0.8992, Val Acc=0.6437
Epoch 10: Train Acc=0.9262, Val Acc=0.6450
```

The CNN model shows a clear divergence between training and validation performance on the CIFAR-10 dataset. While training loss decreases steadily and training accuracy reaches over 90 percent, validation loss begins to increase after a few epochs and validation accuracy plateaus around 65percent. This indicates that the model starts to overfit the training data and fails to generalize to unseen data. Compared to MNIST, CIFAR-10 presents a significantly more complex problem, requiring deeper architectures and more advanced feature extraction. The current CNN structure is not sufficient to capture the variability in CIFAR-10, leading to limited performance. The increase in validation loss alongside decreasing training loss is a strong indicator that the model has entered an overfitting regime.

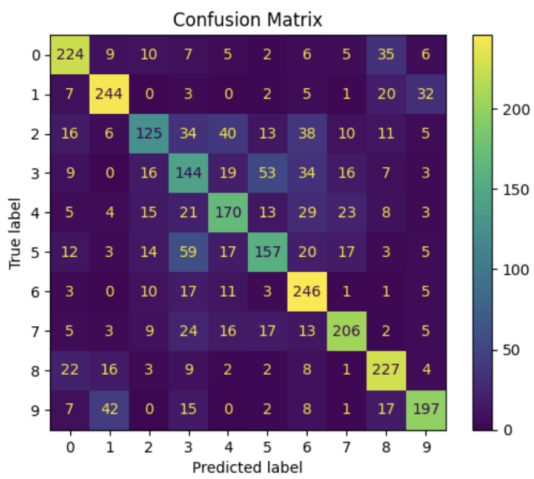


The CNN model achieves an accuracy of approximately 65 percent on the CIFAR, which is significantly higher than both logistic regression and MLP models. However, the performance varies across classes. Some classes such as 0, 1, 7, and 8 show relatively high F1 scores, while others like class 2 and 3 exhibit much lower performance. This indicates that the model struggles to distinguish between certain visually similar categories. Additionally, differences between precision and recall values suggest that the model tends to overpredict some classes while under-detecting others. Overall, the results highlight the increased difficulty of CIFAR-10 compared to MNIST and demonstrate the limitations of the current CNN architecture in capturing complex visual patterns.

	precision	recall	f1-score	support
0	0.72	0.72	0.72	309
1	0.75	0.78	0.76	314
2	0.62	0.42	0.50	298
3	0.43	0.48	0.45	301
4	0.61	0.58	0.60	291
5	0.59	0.51	0.55	307
6	0.60	0.83	0.70	297
7	0.73	0.69	0.71	300
8	0.69	0.77	0.73	294
9	0.74	0.68	0.71	289
accuracy			0.65	3000
macro avg	0.65	0.65	0.64	3000
weighted avg	0.65	0.65	0.64	3000

Macro F1: 0.6430253302577118
Weighted F1: 0.6433048665937309

The confusion matrix of the CNN model on CIFAR-10 shows that although the model correctly classifies a significant number of samples (as seen in the strong diagonal values), there are still considerable misclassifications across several classes. In particular, classes such as 2 and 3 are frequently confused with other categories, indicating that the model struggles to capture fine-grained visual differences. These errors are mainly due to the higher complexity of CIFAR-10 images, which include color variations, complex backgrounds, and similar object structures. Compared to the MNIST confusion matrix, which is nearly perfectly diagonal, the CIFAR-10 results clearly demonstrate the limitations of the current CNN architecture in handling more challenging datasets.



Chapter 3

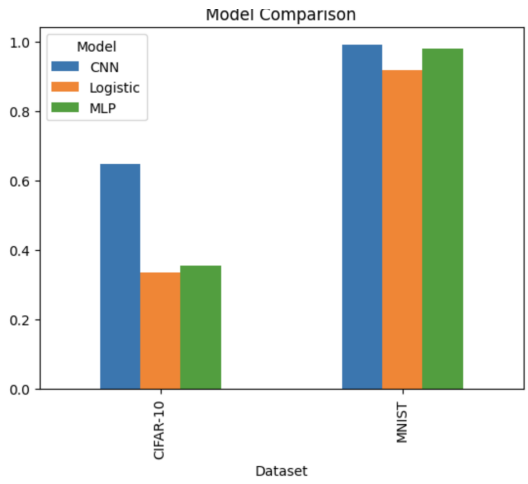
Discussion

The results of this study clearly show that both the choice of model and the complexity of the dataset have a significant impact on classification performance. When comparing the three models, it is evident that the Convolutional Neural Network (CNN) consistently achieves the best results, followed by the Multi-Layer Perceptron (MLP), while Logistic Regression performs the worst across both datasets. This difference can be explained by the ability of each model to capture patterns in image data. Logistic Regression is a linear model and treats each pixel independently, which limits its performance on image classification tasks. The MLP improves upon this by introducing non-linearity through hidden layers, allowing it to learn more complex relationships. However, since it relies on flattened input, it still fails to capture spatial relationships between pixels. In contrast, CNNs preserve the spatial structure of images and are able to extract local and hierarchical features, which leads to significantly better performance.

```
===== FINAL RESULTS =====
    Dataset     Model  Accuracy
0     MNIST  Logistic  0.917714
1     MNIST       MLP  0.979143
2     MNIST       CNN  0.990190
3  CIFAR-10  Logistic  0.335333
4  CIFAR-10       MLP  0.355667
5  CIFAR-10       CNN  0.646667
```

A clear difference can also be observed between the MNIST and CIFAR-10 datasets. The models achieve very high accuracy on MNIST, with CNN reaching nearly perfect performance. This is expected since MNIST consists of simple, grayscale images with low variability. On the other hand, CIFAR-10 is much more complex due to the presence of color images, varying backgrounds, and more diverse object structures. As a result, the performance of all models drops significantly on CIFAR-10, especially for Logistic Regression and MLP. While CNN still performs better than the other models, its accuracy is notably lower compared to its performance on MNIST.

Overall, these results highlight that model performance depends not only on the architecture but also on how well the model aligns with the structure of the data. While simpler models may be sufficient for basic datasets like MNIST, more advanced architectures are required for complex datasets such as CIFAR-10.



Chapter 4

Conclusion

In this study, three different models, Logistic Regression, a Multi-Layer Perceptron (MLP), and a Convolutional Neural Network (CNN), were implemented and evaluated on two benchmark image classification datasets, MNIST and CIFAR-10. The main objective was to compare how different model architectures perform on datasets with varying levels of complexity.

The results show that model performance is strongly influenced by both the structure of the model and the nature of the data. Logistic Regression, as a linear model, provides a basic baseline but performs poorly, especially on more complex data. The MLP improves performance by learning non-linear relationships, but its inability to preserve spatial information limits its effectiveness. The CNN achieves the best results overall, as it is specifically designed to capture spatial patterns and hierarchical features in image data.

A key observation from this study is the clear difference between the two datasets. While all models perform relatively well on MNIST due to its simplicity, their performance drops significantly on CIFAR-10. This demonstrates that CIFAR-10 is a much more challenging dataset and requires more advanced feature extraction capabilities. Although CNN outperforms the other models on CIFAR-10, its performance is still far from perfect, indicating that more sophisticated architectures or additional techniques may be necessary.

The most important conclusion is that choosing an appropriate model architecture is critical for image classification tasks. Simpler models may be sufficient for basic datasets, but more complex data requires models that can effectively capture spatial and structural information. This highlights the importance of matching the model to the problem in order to achieve optimal performance.

Chapter 5

Future Work

The results of this study suggest several possible directions for improving model performance, especially on more complex datasets such as CIFAR-10. First, deeper and more advanced CNN architectures could be explored to better capture hierarchical and high-level features in images. Models with additional convolutional layers or modern architectures such as ResNet may significantly improve performance [1], [9]. Second, data augmentation techniques can be applied to increase the diversity of the training data. Transformations such as rotation, flipping, and cropping help the model generalize better by exposing it to different variations of the same images [3]. Finally, hyperparameter tuning could be further optimized. Parameters such as learning rate, batch size, number of epochs, and dropout rate have a direct impact on model performance and generalization. Systematic tuning or methods such as grid search could lead to better results [5].

Bibliography

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[2] GeeksforGeeks. *Feedforward Neural Network*. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/feedforward-neural-network/. 2025.

[3] GeeksforGeeks. *Pooling Layer in CNN*. [Online]. Available: https://www.geeksforgeeks.org/. 2025.

[4] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd. Sebastopol, CA, USA: O'Reilly Media, 2019.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[6] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization." In: *International Conference on Learning Representations (ICLR)*. 2015.

[7] NVIDIA. *Convolutional Neural Networks*. [Online]. Available: https://developer.nvidia.com/discover/convo
neural-network. 2025.

[8] Nitish Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." In: *Journal of Machine Learning Research* 15 (2014), pp. 1929–1958.

[9] Ryo Yamashita et al. "Convolutional Neural Networks: An Overview and Application in Radiology." In: *Insights into Imaging* 9.4 (2018), pp. 611–629.